



Detecting Chain-of-Thought Poisoning in Enterprise LLM Applications

A Defense-in-Depth Approach for SOC Security Triage

Tendai Nemure

Katz School of Science and Health

Yeshiva University

Supervisor: Professor Dan Galeon

This project was submitted in partial fulfillment of the requirements for the Master of Science degree in Cybersecurity

New York, NY, United States

April 2026

Abstract

Enterprise organizations increasingly deploy Large Language Models (LLMs) for security operations tasks such as incident triage, compliance classification, and threat analysis. These systems often depend on Retrieval-Augmented Generation (RAG) pipelines that incorporate internal runbooks, policy documents, analyst notes, and historical incident data into the model’s reasoning process. This creates a security risk: Chain-of-Thought (CoT) poisoning, where adversarial context corrupts the reasoning pathway that guides the model’s final security decision.

This capstone presents a defense-in-depth detection system for identifying CoT poisoning in enterprise LLM security triage applications. The system combines three complementary layers: regex-based pattern detection for known prompt injection techniques, behavioral drift detection that compares clean and poisoned classifications, and an independent LLM-as-Judge reasoning validator. The layers are combined through an ensemble risk-scoring mechanism designed to flag or block suspicious triage decisions.

The evaluation uses security triage cases derived from CICIDS2017 scenarios, MITRE ATT&CK techniques, and controlled poisoning templates. Results show that realistic poison templates induced behavioral drift in 24.8% of test cases, demonstrating a genuine attack surface without overstating model vulnerability. The final ensemble achieved 95.7% precision, 90.0% recall, and 92.8% F1 score, showing that layered runtime monitoring can substantially improve trust in LLM-assisted SOC workflows.

Lay Summary

This project studies a new security problem created by the growing use of artificial intelligence in cybersecurity operations. Many organizations now use Large Language Models, or LLMs, to help security teams classify alerts, summarize incidents, and decide whether a case should be escalated.

The problem is that these models often rely on internal documents such as runbooks, analyst notes, and policy files. If an attacker poisons those documents, the model may still sound logical while reaching the wrong conclusion. For example, it may downgrade a serious ransomware alert or treat suspicious access as normal activity.

To address this problem, this project builds a defense-in-depth system with three layers. The first layer checks for known attack patterns. The second layer compares how the model behaves with clean context versus suspicious context. The third layer uses a separate model to judge whether the reasoning process appears manipulated.

The results show that realistic poisoning attacks changed the model's decision in 24.8% of tests. This means the attack is realistic, but the model is not easily fooled every time. When the three defense layers were combined, the system achieved 95.7% precision and 90.0% recall. This shows that layered monitoring can help security teams use LLMs more safely in SOC workflows.

Acknowledgements

I would like to thank Professor Dan Galeon for supervising this capstone project and providing guidance throughout the research and development process.

I also acknowledge the support of the Katz School of Science and Health at Yeshiva University and the broader cybersecurity learning community that helped shape my understanding of AI security, SOC workflows, and enterprise risk management.

Contribution Statement

This capstone was completed by Tendai Nemure as part of the CYB 6500 Cybersecurity Capstone. The author designed the project scope, implemented the defense-in-depth architecture, developed the evaluation approach, analyzed the results, and prepared the final report.

The original contribution of this project is the design and evaluation of a practical three-layer runtime detection framework for Chain-of-Thought poisoning in enterprise LLM-based SOC triage systems. The work combines pattern detection, behavioral drift detection, and LLM-as-Judge reasoning validation into a single ensemble risk-scoring pipeline.

Contents

Abstract	1
Lay Summary	2
Acknowledgements	3
Contribution Statement	4
List of Abbreviations	8
1 Analysis of the Initial Problem	9
1.1 Objectives	10
1.2 Assumptions	10
2 Discussion of Improvement Opportunities	12
2.1 Layer 1: Pattern-Based Detection	12
2.2 Layer 2: Behavioral Drift Detection	12
2.3 Layer 3: LLM-as-Judge Reasoning Validation	13
2.4 Evaluation Methodology for LLM-as-Judge	13
2.5 Ensemble Risk Scoring	14
3 Business Case for Identified Improvements	15
3.1 Operational Risk Reduction	15
3.2 SOC Efficiency	15

3.3	Regulatory Compliance	16
4	Project Plan: Scope, Deliverables, and Timeline	17
4.1	Scope	17
4.2	Timeline	17
5	Project Tracking and Success Metrics	19
5.1	Layer 1 Results: Pattern Detection	19
5.2	Layer 2 Results: Behavioral Drift Detection	19
5.3	Detection Rate by Poison Type	20
5.4	Success Criteria Tracking	20
5.5	Evaluation Pipeline	21
6	Project Outcomes and Lessons Learned	23
6.1	Key Outcomes	23
6.2	Lessons Learned	23
6.2.1	Ensemble Design Matters	23
6.2.2	Low-Severity Cases Are Naturally Resistant	24
6.2.3	Model Robustness Should Inform Deployment Design	24
6.3	Project Completion	24
6.4	AI Usage Disclosure	24
A	Supplementary Materials	25
A.1	Demo and Repository Links	25
A.2	Defense-in-Depth Architecture	25

List of Tables

4.1	Project timeline and deliverables	18
5.1	Layer 1 pattern-detection results	19
5.2	Layer 2 behavioral drift results	20
5.3	Detection rate by poison type	20
5.4	Functional requirement tracking	21
5.5	Evaluation pipeline	22

List of Abbreviations

AI	Artificial Intelligence
CoT	Chain-of-Thought
DLP	Data Loss Prevention
FPR	False Positive Rate
IR	Incident Response
LLM	Large Language Model
MTTD	Mean Time to Detect
MTTR	Mean Time to Respond
NIST	National Institute of Standards and Technology
RAG	Retrieval-Augmented Generation
SOC	Security Operations Center
SOAR	Security Orchestration, Automation, and Response

Chapter 1

Analysis of the Initial Problem

Enterprise organizations are rapidly deploying Large Language Models (LLMs) for security operations, including automated incident triage, compliance classification, and threat analysis. These LLM-powered systems typically use Retrieval-Augmented Generation (RAG) to incorporate internal knowledge bases, SOC runbooks, policy documents, and historical incident data into their reasoning process. This architecture creates a critical vulnerability: Chain-of-Thought (CoT) poisoning.

Unlike traditional prompt injection, which targets the input layer with commands such as “ignore previous instructions,” CoT poisoning corrupts the reasoning mechanism itself. An attacker who compromises a RAG knowledge base, internal wiki, SharePoint repository, or vector database can inject documents that mimic legitimate SOC artifacts. Examples include fabricated runbooks citing high false-positive rates, forged CISO policy directives reclassifying alert categories, or manipulated metrics reports with invented statistics. The LLM, instructed to trust its retrieved context as authoritative internal policy, may produce logically coherent but dangerously incorrect security classifications.

The consequences are severe. A poisoned LLM triage system could consistently downgrade ransomware alerts from HIGH to LOW severity, suppress escalation of data exfiltration events, or misclassify unauthorized access as routine activity. Because the LLM’s reasoning appears sound and references “internal policy,” these misclassifications can evade conventional content filters and human review.

Traditional security controls, including firewalls, data loss prevention systems, and web application firewalls, operate primarily at the network and data layers. They are not designed to detect semantic manipulation of reasoning chains. Single-layer defenses such as keyword filtering are also easily evaded by sophisticated poison that uses the language of legitimate

security operations. Enterprise security teams therefore need detection tools and response playbooks specifically designed for LLM reasoning threats.

1.1 Objectives

The objectives of this capstone are as follows:

- Implement pattern-based detection using regex signatures extracted from real prompt injection attacks in the Tensor Trust dataset to catch known CoT poisoning techniques.
- Implement behavioral anomaly detection that flags decision drift by comparing LLM classifications under clean and poisoned context.
- Implement LLM-as-Judge reasoning validation using a separate Claude instance to analyze whether the primary LLM’s reasoning chain contains manipulation indicators.
- Demonstrate defense-in-depth effectiveness for Kaylon Inc. by showing that attacks evading individual detection layers can still be caught by the ensemble.
- Validate system performance through evaluation of precision, recall, F1 score, false positives, and latency across all three layers.
- Align the system with the NIST AI Risk Management Framework.

1.2 Assumptions

The project is based on the following assumptions:

- **Attacker access:** Attackers can inject poisoned documents into the knowledge base but cannot modify the LLM model, system prompts, or detection system code.
- **Model vulnerability:** The LLM is susceptible to Chain-of-Thought poisoning when sufficiently realistic poison is introduced.
- **Ground truth validity:** NIST SP 800-61 incident handling guidelines provide appropriate severity and action labels. Layer 2 drift detection serves as a reliable oracle for labeling reasoning chains in Layer 3 evaluation.

- **Dataset representativeness:** Fifty security triage cases derived from CICIDS2017 network attacks and MITRE ATT&CK techniques adequately represent common SOC workflows and attack patterns.

Chapter 2

Discussion of Improvement Opportunities

This project addresses the identified gap through a defense-in-depth architecture composed of three complementary detection layers. Each layer targets a different attack sophistication level. The approach is modeled on a core cybersecurity principle: no single control is sufficient, and layered defenses provide greater resilience against diverse attack vectors.

2.1 Layer 1: Pattern-Based Detection

The first layer performs sub-millisecond regex-based signature matching using 94 patterns extracted from analysis of 4,000 successful prompt injection attacks in the Tensor Trust dataset. These patterns detect known injection techniques, including instruction overrides, role hijacking, encoded payloads, and authority claims.

Layer 1 functions as a fast pre-filter. It is not designed to provide full semantic understanding. Instead, its purpose is to catch crude or known attacks before they reach deeper and more expensive validation layers.

2.2 Layer 2: Behavioral Drift Detection

The second layer addresses attacks that evade pattern matching by comparing the LLM's classification decisions under clean and suspicious contexts. For each security case, Layer 2 first obtains a baseline decision using the original clean context. It then re-runs the

classification with the potentially poisoned context retrieved from the knowledge base.

When the poisoned context causes the model to downgrade severity, such as HIGH to LOW, or suppress escalation, such as ESCALATE to NO ESCALATE, Layer 2 flags the decision as compromised. This approach detects novel attacks that use no known signatures because it focuses on the behavioral effect of the poison rather than the textual form of the poison.

2.3 Layer 3: LLM-as-Judge Reasoning Validation

The third layer operates as an independent reasoning-integrity classifier. It evaluates whether the primary model’s decision process has been semantically influenced by untrusted or contradictory context. A separate LLM instance receives structured inputs: the original alert, the retrieved context, and the primary model’s reasoning trace. It then evaluates the reasoning for indicators of compromise, including unsourced statistics, embedded directives disguised as policy, logical inconsistencies, and conclusions that contradict the alert evidence.

The judge is architecturally isolated from the primary model. This separation reduces shared failure modes and improves independence in the validation process.

2.4 Evaluation Methodology for LLM-as-Judge

The LLM-as-Judge was evaluated as a binary reasoning-integrity classifier. A labeled dataset of reasoning chains was constructed using the Layer 2 evaluation framework, which produces baseline and poisoned decisions with full reasoning traces for each case.

Ground truth labels were assigned based on Layer 2 drift detection. Reasoning chains where Layer 2 detected behavioral drift, such as severity downgrade or escalation suppression, were labeled COMPROMISED. Reasoning chains with no detected drift were labeled CLEAN. This approach ensures that labels reflect actual reasoning manipulation rather than merely the presence of poisoned content.

The judge was evaluated on 60 reasoning chains: 10 cases across six conditions, consisting of one clean condition and five poison types. Standard classification metrics were used, including precision, recall, F1 score, false-positive rate on clean cases, and per-query latency. Adversarial robustness was also assessed by testing whether sophisticated poison that evades Layers 1 and 2 can deceive the judge.

2.5 Ensemble Risk Scoring

Signals from all three layers feed into a combined risk-scoring engine using tiered OR logic. If any single layer fires, the decision is flagged as medium risk. If two or more layers agree, the decision is blocked as high risk. This design prevents one silent layer from neutralizing a strong signal from another layer and implements defense-in-depth at the decision level.

Chapter 3

Business Case for Identified Improvements

For Kaylon Inc., the fictional enterprise security company used as this project’s case study, the business case for a CoT poisoning detection system rests on three pillars: operational risk reduction, SOC efficiency, and regulatory compliance.

3.1 Operational Risk Reduction

A compromised triage LLM that suppresses escalation of critical incidents directly increases mean time to respond (MTTR) and mean time to detect (MTTD). Experimental results in this project show that document-style CoT poisoning successfully manipulated LLM triage decisions in 24.8% of test cases using realistic, calibrated attack templates. A single missed ransomware escalation or suppressed data exfiltration incident could expose an organization to financial loss, operational disruption, and regulatory penalties.

3.2 SOC Efficiency

The detection system operates as an inline gateway that adds minimal latency to the triage pipeline. Layer 1 adds less than 1 ms. In production, Layer 2 baseline decisions can be cached, reducing per-query overhead to a single API call. Rather than replacing analyst judgment, the system provides automated confidence scoring that helps analysts prioritize which LLM decisions to trust and which to manually verify.

3.3 Regulatory Compliance

The system is designed to align with the NIST AI Risk Management Framework, particularly controls related to AI system monitoring, risk response, and incident tracking. As AI governance requirements mature, organizations deploying LLMs in security-critical roles will need auditable mechanisms for detecting and responding to reasoning manipulation. This project provides a practical control framework for that requirement.

Chapter 4

Project Plan: Scope, Deliverables, and Timeline

4.1 Scope

The project delivers a Python-based LLM Security Gateway that monitors and validates LLM security triage decisions in real time using a defense-in-depth architecture. The dataset is constructed using a semi-automated pipeline combining real-world prompt injection datasets, security incident patterns from CICIDS2017, and controlled poisoning templates.

Existing adversarial prompts are transformed into domain-specific CoT poisoning scenarios targeting security triage workflows. Each case is paired with ground truth severity and escalation labels to enable automated evaluation. The project scope includes five poison attack types: deescalation bias, false positive framing, authority hijacking, policy contradiction, and statistical manipulation.

The implementation includes three detection layers, a FastAPI integration gateway, ensemble orchestration, evaluation metrics, and NIST AI RMF alignment documentation.

4.2 Timeline

Table 4.1: Project timeline and deliverables

Phase	Weeks	Deliverables	Status
Foundation	1–2	Proposal, repository structure, Tensor Trust analysis	Complete
Layer 1	3–4	Pattern library, evaluation framework	Complete
Layer 2	5–6	Behavioral detector, poison injector, drift analysis	Complete
Risk Scoring	7–8	Combined risk scorer, dataset expansion	Complete
Layer 3	8–9	LLM-as-Judge validator, reasoning-chain evaluation	Complete
Integration	10	FastAPI gateway, ensemble orchestration	Complete
Evaluation	10	Metrics suite and confusion matrices	Complete
Documentation	11	Technical report, NIST mapping, SOC operations guide	Complete
Final	–	Presentation, demo, code submission	Complete

Chapter 5

Project Tracking and Success Metrics

5.1 Layer 1 Results: Pattern Detection

Layer 1 was evaluated against 4,000 randomly sampled attacks from the Tensor Trust dataset. Detection reached 58.6%, with average latency of 0.14 ms and a false-positive rate below 2% on clean security cases. Although the detection rate fell below the standalone 70% target, this result is acceptable within the ensemble design because Layer 1 is intended to catch known patterns quickly rather than provide comprehensive coverage.

Table 5.1: Layer 1 pattern-detection results

Metric	Result	Target	Status
Detection rate	58.6%	$\geq 70\%$	Below target; ensemble acceptable
Average latency	0.14 ms	< 10 ms	Pass
False-positive rate	$< 2\%$	$< 2\%$	Pass

5.2 Layer 2 Results: Behavioral Drift Detection

Layer 2 was evaluated on 50 security triage cases derived from three sources: 20 cases adapted from CICIDS2017 network attack scenarios, 20 cases based on MITRE ATT&CK techniques, and 10 edge or ambiguous cases. All cases were labeled with ground truth severity and action, validated against NIST SP 800-61 incident handling guidance.

Each case was tested against five poison types, producing 250 total test instances. The overall drift detection rate was 24.8%, or 62 of 250 tests. This result reflects the use of

calibrated poison templates designed to sound like genuine SOC documents, analyst notes, SOAR enrichment summaries, and incident response runbook entries. The model resisted 75% of realistic poison attempts, showing that the evaluated model is not trivially fooled by subtle context manipulation. The 24.8% that did succeed represents the realistic attack surface the ensemble must defend against.

Table 5.2: Layer 2 behavioral drift results

Metric	Result	Target	Status
Overall drift detection rate	24.8% (62/250)	–	Realistic baseline
Average latency	Approx. 4.10 s	<5 s	Expected
Escalation suppression	Subset of 62 drifts	–	Key danger signal

5.3 Detection Rate by Poison Type

Performance varied by attack strategy. Narrative-style attacks that provide a specific alternative explanation, such as deescalation bias and authority hijacking, were most effective at inducing drift. Procedural formats and statistical summaries were more easily resisted.

Table 5.3: Detection rate by poison type

Poison Type	Tests	Drift Detected	Detection Rate
Deescalation bias	50	19	38.0%
Authority hijacking	50	18	36.0%
False positive framing	50	16	32.0%
Statistical manipulation	50	6	12.0%
Policy contradiction	50	3	6.0%
Total	250	62	24.8%

5.4 Success Criteria Tracking

Table 5.4: Functional requirement tracking

ID	Requirement	Status
FR1	Structured input processing: 100% parse rate	Complete; 10/10 cases parsed
FR2	Primary LLM classification: valid JSON output	Complete; 100% valid responses
FR3	Pattern detection: $\geq 70\%$ detection, 94 signatures	Complete; 58.6%, ensemble acceptable
FR4	Behavioral detection across five poison types	Complete; 24.8% drift
FR5	LLM-as-Judge: $\geq 80\%$ accuracy	Complete; precision 97.4%, recall 76.0%, F1 85.4%
FR6	Risk scoring: ensemble confusion matrix	Complete; precision 95.7%, recall 90.0%, F1 92.8%
FR8	Poison injection: five or more types measurably change decisions	Complete; five types, 24.8% effect rate
FR10	Ensemble: precision $\geq 90\%$, recall $\geq 88\%$	Complete; precision 95.7%, recall 90.0%

5.5 Evaluation Pipeline

A standardized six-step pipeline is used to evaluate each detection layer for LLM poisoning.

Table 5.5: Evaluation pipeline

Step	Stage	Description
1	Case selection	Labeled dataset with ground truth severity and response actions aligned to NIST SP 800-61.
2	Baseline generation	Each case is processed under clean conditions to establish reference decisions and reasoning traces.
3	Poison injection	Five adversarial poison types generate 250 total test instances.
4	Layered detection	Three independent layers analyze context, decision drift, and reasoning integrity.
5	Metric computation	Performance is measured using detection rate, precision, recall, F1 score, false positives, and latency.
6	Ensemble analysis	Cross-layer results are compared to assess complementary detection coverage and residual vulnerabilities.

Chapter 6

Project Outcomes and Lessons Learned

6.1 Key Outcomes

The project demonstrated that Chain-of-Thought poisoning is a viable and dangerous attack vector against enterprise LLM triage systems while also showing that modern LLMs are not trivially fooled by subtle manipulation. Using calibrated poison templates that mimic genuine SOC documents, the system induced behavioral drift in 24.8% of test cases. This demonstrates a genuine attack surface without overstating model vulnerability.

The three-layer ensemble closes this gap effectively. Layer 1 catches known pattern-based injections in under 1 ms. Layer 2 catches semantic drift that pattern matching misses. Layer 3 audits reasoning chains independently and achieved 97.4% precision on flagged cases. The ensemble achieved 95.7% precision and 90.0% recall across 60 evaluation chains, meaning most real attacks are caught while false alarms remain low.

6.2 Lessons Learned

6.2.1 Ensemble Design Matters

An initial weighted-average scoring approach,

$$0.25L_1 + 0.45L_2 + 0.30L_3,$$

produced only 20% recall because a strong Layer 1 signal could be canceled out when Layer 2 and Layer 3 returned zero. Switching to OR logic, where any single layer firing triggers a flag and two or more layers trigger a block, better implements defense-in-depth and raised ensemble recall to 90%.

6.2.2 Low-Severity Cases Are Naturally Resistant

Cases with ground truth severity of Low showed lower drift because the baseline classification already reflected low priority. This means the system has a naturally low false-positive rate on low-severity cases, which is operationally desirable in a SOC context where low-priority alerts should flow through without unnecessary analyst friction.

6.2.3 Model Robustness Should Inform Deployment Design

The evaluated model resisted approximately 75% of realistic poison attempts, which is encouraging. However, the 25% that succeeded represent real operational risk. The ensemble's 90% recall shows that model-level robustness is not sufficient on its own; runtime monitoring remains necessary.

6.3 Project Completion

All planned deliverables were completed, and the project objectives were achieved. The final system includes the three detection layers, ensemble scoring, evaluation pipeline, technical documentation, NIST alignment, and presentation-ready results.

6.4 AI Usage Disclosure

AI tools assisted with generating architecture diagrams and building a semi-automated pipeline to transform CICIDS2017 network logs into natural language triage case descriptions. All cases were manually validated and labeled.

Chapter A

Supplementary Materials

A.1 Demo and Repository Links

- **Demo link:** <https://www.loom.com/share/a6ef692b05a64045a01bf8a276d42946>
- **Source code:** <https://github.com/picassoendless/cot-poisoning-detection>
- **Network Intrusion dataset (CIC-IDS-2017):** <https://www.unb.ca/cic/datasets/ids-2017.html>
- **Tensor Trust dataset:** <https://tensortrust.ai/>

A.2 Defense-in-Depth Architecture

The final implementation uses three complementary detection layers:

1. **Layer 1: Context inspection** through regex-based prompt injection and poisoning signatures.
2. **Layer 2: Model behavior inspection** through baseline-versus-poisoned decision drift.
3. **Layer 3: Reasoning inspection** through independent LLM-as-Judge validation.

These signals are aggregated through tiered OR logic. A single positive layer produces a flag, while two or more positive layers produce a block decision.

Bibliography

- [1] A. Habibzadeh et al., “On the dangers of poisoned LLMs in security automation,” arXiv preprint, 2025.
- [2] S. Zhao et al., “Chain-of-thought hijacking in LLM applications,” arXiv preprint, 2025.
- [3] H. Lehman et al., “Medical large language models are vulnerable to data-poisoning attacks,” *Nature Medicine*, 2025.
- [4] Anthropic, “A small number of samples can poison LLMs of any size,” Anthropic Research, 2023.
- [5] S. Toyer, O. Watkins, A. Mendes, J. Leech, M. Lewis, C. Denison, T. Henighan, T. Hume, and A. Freire-Morán, “Tensor Trust: Interpretable prompt injection attacks from an online game,” *Proceedings of the 37th Conference on Neural Information Processing Systems*, 2024.
- [6] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pp. 108–116, 2018.
- [7] MITRE Corporation, “MITRE ATT&CK Framework,” 2024. Available: <https://attack.mitre.org/>
- [8] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, 2023.
- [9] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Revision 2, 2012.
- [10] Microsoft Security Response Center, “How Microsoft defends against indirect prompt injection attacks,” 2025.

- [11] Prompt Security, “The embedded threat in your LLM: Poisoning RAG pipelines via vector embeddings,” 2025.
- [12] FuzzingLabs, “Attacking reasoning models: LLM vulnerabilities,” 2025.
- [13] SANS Institute, “SANS 2024 detection and response survey,” 2026.